



전자전기컴퓨터설계실험Ⅱ

실험 전 보고서

LAB3

학과	전자전기컴퓨터공학부
이름 / 학번	김승민 / 2023440021
작성일	2026년 09월 26일
대상	FPGA 응용회로 실습
검증 환경	Vivado XSim 2026.1 / VS Code VaporView

1. 실험 목적과 관련 이론

실험 목적

FPGA 응용 회로 7개를 VS Code에서 먼저 검증하고 Vivado와 보드에 확인한다. PWM, 구동기, 표시장치, 직렬통신을 50 MHz 단일 클록으로 구현한다.

관련 이론

- 50 MHz 단일 클록

최상위 입력 clk_50mhz는 MAIN CLOCK F, 50 MHz다. 클록 주기는 20ns다. LED PWM, 모터 step, 1초 tick, LCD tick, UART bit timing은 counter가 만드는 clock-enable로 제어한다. 주파수를 parameter로 받는 하위 모듈은 일반 이름 clk를 쓸 수 있다. 분주한 일반 신호는 다른 always 블록의 clk로 사용하지 않는다.

- 파라미터 계산

기능	기준 계산
PWM 주기	CLK_HZ/PWM_HZ
피에조 반주기	CLK_HZ/(2TONE_HZ)
모터 step 간격	CLK_HZ/STEP_HZ
1초 tick	CLK_HZ 클록마다 1회
UART bit	반올림한 CLK_HZ / BAUD

나눗셈 결과와 counter 범위를 계산한 뒤 parameter가 0이 되는 조건을 피한다.

- 비동기 입력 처리

버튼, UART RX는 FPGA clk와 무관한 시점에 바뀐다. 첫 두 플립플롭으로 동기화한다. 기계식 버튼은 동기화 뒤 안정 시간을 확인하고 눌림당 한 클록 펄스를 만든다. 방향, enable 같은 레벨 입력도 두 단계 동기화를 거친다. 시뮬레이션은 메타안정성 자체가

아니라 동기화 이후의 디지털 순서를 검사한다.

실행 방법

1. v2.0.2 빈 템플릿 clone하고 자신의 main 브랜치 생성한다.
2. RTL-TB-XDC-simulation.json 직접 작성한다.
3. VS Code에서 01 Check tools → 02 Simulate → 03 Open waveform을 실행한다.
4. Vivado GUI에서 같은 원본으로 XSim → 합성 → 구현 → bit 생성한다.
5. Vivado를 보드와 연결하고 보드에서 시뮬레이션을 실행한다.

실행 환경



- GitHub 계정 주소 기재

계정: thechef030

주소: <https://github.com/thechef030>

공통된 도구 버전

icarus verilog: version 13.0

git: version 2.55.0.windows.5

python: python 3.13.15

vscode: 1.137.0

2. 실험 1 PWM LED 밝기

회로 목적

버튼을 누를 때마다 PWM(Pulse Width Modulation)의 듀티비를 10%씩 증가시키고, 동일한 PWM 신호를 8개의 LED에 출력한다. 주기는 일정하게 유지하면서 HIGH인 시간의 비율을 바꾸어 평균 밝기를 조절한다.

블록 흐름과 동작 원리

button → 2단 동기화 → 디바운스 → 1클록 press → level → PWM 비교기 → led[7:0]

버튼이 안정적으로 눌렸을 때만 press를 한 클록 동안 발생시킨다. level은 0부터 10까지 증가한 뒤 0으로 돌아간다. PWM 카운터와 threshold를 비교하여 $count < threshold$ 일 때 HIGH를 출력한다. 출력은 레지스터에서 갱신되므로 클록 상승 직후의 값을 기준으로 관찰한다.

파라미터 계산

항목	계산과 의미
PWM 주기	$50,000,000 / 1,000 = 50,000$ 클록 = 1 ms
듀티 단계	$threshold = 50,000 \times level / 10$, 한 단계 10%
카운터 폭	PWM 16비트, level 4비트, 디바운스 20비트
버튼 안정 시간	$1,000,000 \times 20 \text{ ns} = 20 \text{ ms}$
TB 가속 조건	주기 10클록 = 200 ns, 디바운스 2클록

상태와 타이밍

조건	동작
리셋	level=0, pwm=0
press=1	0 → 1 → ... → 10 → 0
level=0 / 3 / 10	듀티 0% / 30% / 100%

RTL과 TB 및 XDC의 역할

lab3_led_pwm.v가 합성 top이며 버튼 처리와 PWM 모듈을 연결한다.

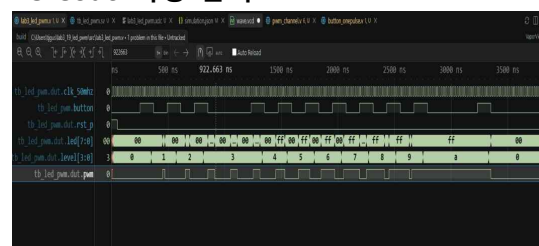
button_onepulse.v는 동기화, 디바운스, 원펄스를 담당하고 pwm_channel.v는 주기 계수와 듀티 비교를 담당한다.

tb_led_pwm.sv의 simulation top은 tb_led_pwm이고 HIGH 클록 수를 검사한다. lab3_led_pwm.xdc는 핀과 20 ns 클록 제약을 지정한다.

TB 자극과 기대 결과

입력과 검사 조건	기대 결과
리셋 후	HIGH 0/10클록
버튼 3회	HIGH 3/10클록
추가 7회	HIGH 10/10클록
추가 1회	HIGH 0/10클록으로 순환

VS Code 파형 분석



위의 Vs Code 파형에서 level은 0, 1, 2, ..., 9, a, 0으로 변한다. a는 16진수로 10을 뜻한다. level 증가에 따라 pwm의 HIGH 폭이 넓어지고, level=10에서는 HIGH를

유지한 뒤 다음 버튼 입력에서 LOW로 돌아간다. led는 모든 비트가 같은 PWM이므로 00과 ff로 표시된다. 버튼 입력과 level 갱신 사이의 지연은 동기화, 디바운스, 원펄스 처리 과정에 해당한다.

또한 위의 파형을 보면 button이 3회 연속으로 눌리고 그 다음 7회 연속으로 눌리는 것을 확인할 수 있는데 이는 TB자극에 의한 것임을 알 수 있다.

Icarus PASS 실행 기록



수정 실험과 복구 결과

tb_led_pwm.v의 DUT 인스턴스에서 .LEVELS(10)을 .LEVELS(5)로 한 항목만 변경했다. PWM 주기는 유지되며, 버튼 3회 입력 후 듀티가 30%에서 60%로 증가할 것으로 예상하였다. TB의 기존 기대값은 바뀌지 않아 차이를 검출하도록 하였다.

구분	결과
예상	10클럭 중 HIGH 3개 → 6개, 단계 폭 10% → 20%
수정 실행	1,231 ns에서 level=3 high=6으로 FAIL. 기존 30% 기대값과의 차이를 검출하였다.
복구 실행	LEVELS=10 복구 후 LAB3_LED_PWM_PASS checks=4; 종료 3,831 ns. 기존 실행과 일치.

보드에서 확인할 입력과 출력

clk_50mhz=B6, rst_p=K4, button=N8을 사용한다. led[0]부터 led[7]까지

N5/M1/M3/M7/N7/M2/M4/L4에 연결한다. 버튼을 길게 눌러도 한 단계만 바뀌는지, 8개 LED의 밝기가 함께 변하는지, 최대 밝기 다음에 모두 꺼지는지 확인한다. 밝기 체감은 듀티비에 정확히 선형일 필요는 없다.

3. 실험 2 RGB LED PWM

회로 목적

R, G, B 버튼으로 세 색의 듀티비를 독립적으로 조절하고 네 개의 RGB LED에 동일한 혼합색을 출력한다. 각 채널의 평균 점등 비율이 색 혼합에 미치는 영향을 확인한다.

블록 흐름과 동작 원리

R·G·B 버튼 → 채널별 원펄스 → level_r/g/b → PWM 3개 → led_r/g/b[3:0]
세 채널은 같은 50 MHz 클럭과 PWM 주기를 사용하지만 밝기 레지스터는 각각 갖는다. 한 색의 버튼을 눌러도 다른 색의 level은 유지된다. 각 색의 PWM을 네 번 복제하므로 네 RGB LED는 같은 색 비율을 갖는다.

파라미터 계산

항목	계산과 의미
공통 PWM 주기	50,000클럭 = 1 ms, 1 kHz
R 20%	HIGH 10,000클럭 = 0.2 ms
G 50% / B 80%	HIGH 25,000 / 40,000클럭 = 0.5 / 0.8 ms
TB 가속 조건	주기 10클럭, HIGH 수 R/G/B = 2/5/8
버튼 처리	채널마다 20 ms 디바운스, 4비트 level

상태와 타이밍

조건	동작
리셋	level_r/g/b=0/0/0
각 버튼 입력	해당 채널만 0~10 순환
R만 추가 입력	2/5/8 → 3/5/8

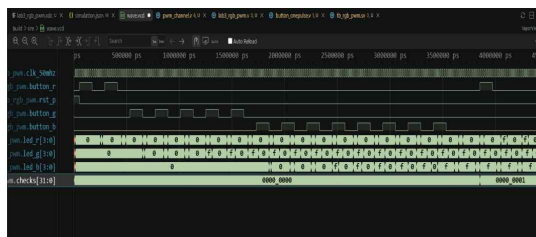
RTL과 TB 및 XDC의 역할

lab3_rgb_pwm.v가 합성 top이다.
button_onepulse.v 3개가 버튼을 처리하고
pwm_channel.v 3개가 독립 듀티를 만든다.
tb_rgb_pwm.sv의 simulation top은
tb_rgb_pwm이며 HIGH 수와 채널
독립성을 검사한다. lab3_rgb_pwm.xdc가
버튼 3개와 RGB 출력 12개 핀을 연결한다.

TB 자극과 기대 결과

입력과 검사 조건	기대 결과
R 2회, G 5회, B 8회	HIGH 수 2/5/8
R 1회 추가	HIGH 수 3/5/8
G-B 추가 입력 없음	G와 B 듀티 유지

VS Code 파형 분석



위의 VS Code 파형은 R, G, B 버튼에
순서대로 자극을 주는 모습을 보여준다. 각
출력은 0 또는 f로 표시되며 f는 해당
색의 4개 LED가 동시에 HIGH임을 뜻한다.
뒤쪽의 3851ns에 R 버튼 추가 입력은 R
채널만 바꾸는 검사에 해당한다. 화면

측척상 R의 짧은 펄스는 숫자 0 사이의
가는 구간으로 보이므로 숫자 라벨만으로
R이 계속 꺼져 있다고 판단하면 안 된다.
재실행에서는 HIGH 수 2/5/8 및 3/5/8을
확인하였다.

Icarus PASS 실행 기록

```

Icarus Verilog 6.21.0
Parsing task in folder /home/.../lab3_rgb_pwm
...
[... 실행 로그 ...]
[... PASS 실행 결과 ...]

```

수정 실험과 복구 결과

lab3_rgb_pwm.v의 리셋문에서 level_r <= 0만 level_r <= 1로 변경했다. 같은 버튼 자극을 가하면 R은 30%가 되고 G와 B는 50%, 80%를 유지할 것으로 예상하였다. 다른 채널과 TB 기대값은 유지했다.

구분	결과
예상	첫 측정 HIGH 수 2/5/8 → 3/5/8
수정 실행	3,851 ns에서 rgb high=3,5,8으로 FAIL. R만 한 단계 증가하였다.
복구 실행	R 초기 level=0 복구 후 LAB3_RGB_PWM_PASS checks=2; 종료 4,431 ns. 기준 실행과 일치.

보드에서 확인할 입력과 출력

버튼 R/G/B는 N8/N4/N1이다. 각 출력의 [3:0] 핀 순서는 R=T2/U1/P2/R3, G=U5/V1/R7/T6, B=U3/W2/R5/T3이다. 한 버튼을 누를 때 해당 색 성분만 변하고 네 LED가 동일한 색으로 보이는지 확인한다. 실제 혼합색은 채널별 광효율에도 영향을 받는다.

4. 실험 3 피에조 단일 음

회로 목적

피에조 출력에 일정한 주기의 구형파를 인가하여 단일 음을 발생시킨다. 50 MHz 입력 클록을 계수하여 목표 음의 반주기마다 출력을 반전시키는 원리를 이해한다.

블록 흐름과 동작 원리

50 MHz 클록 → 반주기 카운터 → 종료값 비교 → piezo 반전

count가 HALF_PERIOD-1에 도달하면 0으로 초기화하면서 piezo를 반전한다. HIGH와 LOW가 각각 한 반주기씩 지속되므로 출력 주파수는 입력 클록을 HALF_PERIOD의 두 배로 나눈 값이다. 반주기마다 토글되는 출력 자체를 내부 로직의 새 클록으로 사용하지 않는다.

파라미터 계산

항목	계산과 의미
목표 주파수	TONE_HZ = 294 Hz
반주기 계수	$\text{floor}(50,000,000 / (2 \times 294))$ = 85,034
실제 계산 주파수	$50,000,000 / 170,068$ ≈ 294.000047 Hz
주기와 카운터	3.40136 ms, count 폭 17비트
TB 가속 조건	CLK_HZ=1000, TONE_HZ=100 → 반주기 5클록

상태와 타이밍

조건	동작
리셋	count=0, piezo=0
종료 전	count 1씩 증가

조건	동작
종료값 도달	count=0, piezo 반전

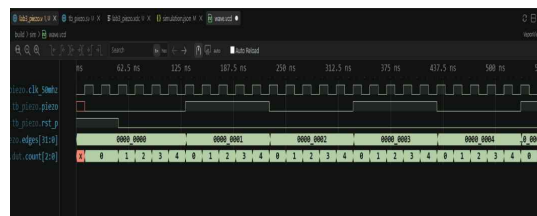
RTL과 TB 및 XDC의 역할

lab3_piezo.v는 합성 top으로 반주기 계수와 출력 반전을 수행한다. tb_piezo.sv의 simulation top은 tb_piezo이며 반전 간격을 5클록과 비교한다. lab3_piezo.xdc는 piezo 출력 핀 및 클록 리셋 제약을 설정한다.

TB 자극과 기대 결과

입력과 검사 조건	기대 결과
리셋 인가	piezo=0
리셋 해제 후 계수	count 0~4 반복
연속 반주기 관찰	5클록마다 반전, edges=5

VS Code 파형 분석



파형에서 count가 0, 1, 2, 3, 4로 반복되고 종료 시점마다 piezo가 반전한다. 입력 클록 주기가 20 ns이므로 시뮬레이션의 반주기는 100 ns, 한 주기는 200 ns이다. edges는 5까지 증가한다. 이 시간은 가속 TB의 결과이며 보드에서 발생시킬 294 Hz의 주기와 구분해야 한다. 맨 처음의 X는 초기화 전 값이며 리셋 적용 뒤에는 유효한 상태가 된다.

Icarus PASS 실행 기록



수정 실험과 복구 결과

tb_piezo.sv의 .TONE_HZ(100)만 .TONE_HZ(50)으로 변경했다. 반주기 계수는 5에서 10으로 늘어 출력 주파수가 절반이 될 것으로 예상하였다. 원래의 5클럭 기대값을 유지하여 변화 검출 여부를 확인했다.

구분	결과
예상	반주기 5 → 10클록, 100 → 200 ns
수정 실행	431 ns에서 half period=10으로 FAIL. 반주기 증가를 검출하였다.
복구 실행	TONE_HZ=100 복구 후 LAB3_PIEZO_PASS edges=5; 종료 551 ns. 기준 실행과 일치.

보드에서 확인할 입력과 출력

piezo 출력은 Y21이며 공통 클록과 리셋은 B6, K4이다. 정상 설정에서 약 294 Hz의 음이 지속되는지 확인한다. 보드에서 TONE_HZ를 낮추면 음 높이가 낮아질 것으로 예상하며 음량 변화는 부품 특성에 따라 관찰한다. 소리의 실제 음높이·음량 확인은 보드 실험에서 수행한다.

5. 실험 4 STEPPER

회로 목적

4개의 코일 구동 패턴을 정해진 순서로 바꾸어 스텝모터의 정방향·역방향 회전과

정지 상태를 제어한다. 입력 동기화, 상태 전이와 step 간격의 관계를 확인한다.

블록 흐름과 동작 원리

enable-direction → 2단 동기화 → step
카운터 → 2비트 state → 4상 패턴
enable_sync가 1일 때 step 간격을
계수한다. 종료값에서 direction_sync
=0이면 state를 증가시키고 1이면 감소시
킨다. 2비트 상태는 0~3을 순환한다.
enable_sync=0이면 카운터만 0으로
만들고 state를 유지하므로 코일 출력도
마지막 패턴을 유지한다.

파라미터 계산

항목	계산과 의미
기본 step 속도	STEP_HZ=100 → 100 step/s
step 간격	$50,000,000 / 100 =$ 500,000클록 = 10 ms
카운터와 상태 폭	count 19비트, state 2비트
TB 가속 조건	CLK_HZ=8, STEP_HZ=2 → 4클록 = 80 ns
패턴 한 순환	4 step = 40 ms, 모터 한 회전과는 다름

상태와 타이밍

조건	동작
state 0 / 1	0011(3) / 0110(6)
state 2 / 3	1100(c) / 1001(9)
정방향 / 역방향	0→1→2→3→0 / 0→3→2→1→0
enable_sync=0	상태 유지, 카운터 초기화

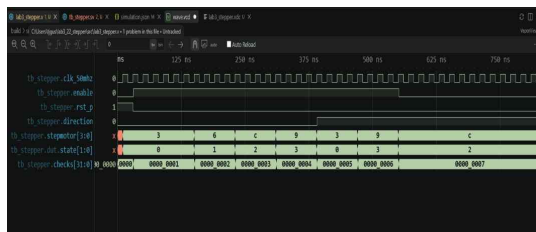
RTL과 TB 및 XDC의 역할

lab3_stepper.v는 합성 top이며 입력 동기화, 간격 계수, 상태 디코딩을 수행한다. tb_stepper.sv의 simulation top은 tb_stepper로 정방향 1순환, 역방향 2단계, 정지 유지를 검사한다. lab3_stepper.xdc는 4상 출력과 입력 핀을 지정한다.

TB 자극과 기대 결과

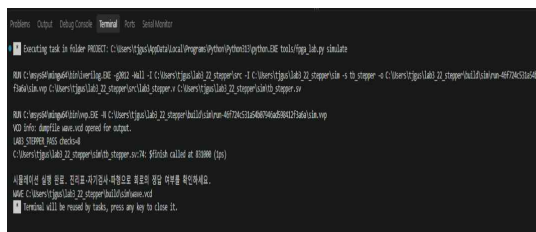
입력과 검사 조건	기대 결과
리셋 및 정방향	$3 \rightarrow 6 \rightarrow c \rightarrow 9 \rightarrow 3$
direction=1	$3 \rightarrow 9 \rightarrow c$
enable=0	동기화 이후 c 유지

VS Code 파형 분석



stepmotor가 3, 6, c, 9, 3으로 변하며
정방향 순서를 따른다. direction을 1로
바꾸면 3, 9, c의 역순 전이가 나타난다.
enable을 0으로 한 뒤에는 c가 유지된다.
VS Code 화면의 checks는 7까지 보이고
정지 검사가 끝나는 최종 시점은 화면
밖이므로, 최종 8회 PASS는 별도 재실행
로그로 확인하였다.

Icarus PASS 실행 기록



수정 실험과 복구 결과

tb_stepper.sv의 .STEP_HZ(2)만 .STEP_HZ(1)로 변경했다. step 간격이 두 배가 되어도 패턴 순서는 같을 것으로 예상하였다. TB는 상태 도달을 기다린 뒤 패턴을 검사하므로 변경 후에도 PASS할 수 있다.

구분	결과
예상	step 간격 4 → 8클록, 80 → 160 ns
수정 실행	LAB3_STEPPER_PASS checks=8. 종료 시각 831 → 1,311 ns, 상태 순서 유지.
복구 실행	STEP_HZ=2 복구 후 LAB3_STEPPER_PASS checks=8; 종료 831 ns. 기준 실행과 일치.

보드에서 확인할 입력과 출력

enable=N8, direction=N4이며
stepmotor[3:0]은 Y20/Y22/AA20/AA21이다.
드라이버 결선 후 회전 방향, 속도, 정지
상태 유지와 발열을 확인한다.

enable=0은 모든 코일을 끄는 명령이 아니므로 정지 중에도 여자 상태가 유지될 수 있다. 회전수 계산에는 모터의 step 각도와 감속비가 추가로 필요하다. 전원을 끈 상태에서 드라이버, 별도 모터 전원, 공통 GND를 연결하고 FPGA 핀에서 모터를 직접 구동하지 않는다.

6. 실험 5 MM SS Counter

회로 목적

1초마다 BCD 카운터를 증가시켜
00:00부터 59:59까지 표시한 뒤 00:00으로
순환한다. 분·초 자리올림과 4자리
7세그먼트의 동적 스캐를 이해한다.

블록 흐름과 동작 원리

50 MHz → 1초 계수 → 분·초 BCD → 세그먼트 디코더 → 자리 선택 스캔
초의 일의 자리와 분의 일의 자리는 0~9, 십의 자리는 0~5로 계수한다. 하위 자리가 최대값에서 0으로 돌아갈 때 상위 자리를 증가시킨다. 표시 회로는 한 번에 한 자리만 선택하고 빠르게 순환한다. 분의 일의 자리에는 구분용 점을 함께 출력한다.

파라미터 계산

항목	계산과 의미
1초 계수	50,000,000클럭, 카운터 26비트
스캔 간격	$50,000,000 / 4,000 = 12,500\text{클럭} = 250\ \mu\text{s}$
한 화면 갱신	$4\text{자리} \times 250\ \mu\text{s} = 1\ \text{ms}$, 자리별 듀티 약 25%
전체 계수 범위	$60\text{분} \times 60\text{초} = 3,600\text{초}$
TB 가속 조건	$\text{CLK_HZ}=2 \rightarrow$ 논리적 1초당 40 ns

상태와 타이밍

조건	동작
일반 증가	00:08 → 00:09
초 자리올림	00:09 → 00:10
분 자리올림	00:59 → 01:00
최대값 순환	59:59 → 00:00

RTL과 TB 및 XDC의 역할

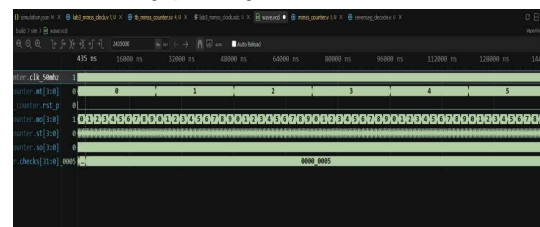
lab3_mmss_clock.v가 합성 top으로 계수기, 디코더, 스캔을 연결한다.

mmss_counter.v는 시간 계수, sevenseg_decode.v는 숫자별 패턴을 만든다. tb_mmss_counter.sv의 simulation top은 tb_mmss_counter이며 계수기와 디코더를 검사한다. 전체 top의 자리 스캔은 이 TB의 직접 검사 대상이 아니다. XDC는 표시장치 핀과 클럭을 지정한다.

TB 자극과 기대 결과

입력과 검사 조건	기대 결과
숫자 디코딩	0=fc, 9=f6
10초 및 추가 50초	00:10, 01:00
추가 3,539초 및 1초	59:59, 00:00

VS Code 파형 분석



파형에서 mt는 0부터 5까지 천천히 변하고, mo는 각 구간에서 0~9를 반복한다. st와 so는 더 빠르게 변하므로 전체 시간을 축소한 화면에서 촘촘하게 겹쳐 보인다. 따라서 so가 녹색 띠로 보인다고 값이 고정된 것은 아니다. 제공 화면의 checks=5는 중간 검사 상태이다. 재실행 로그에서 전체 3,600초 순환과 디코더를 포함한 checks=7 PASS를 확인하였다.

Icarus PASS 실행 기록



수정 실험과 복구 결과

tb_mmss_counter.sv의 .CLK_HZ(2)만 .CLK_HZ(4)로 변경했다. 실제 TB 클록은 20 ns로 유지했다. 기존 advance 함수는 논리적 1초당 2클록을 진행하므로 같은 검사 시점의 표시 시간이 절반만 증가할 것으로 예상하였다.

구분	결과
예상	논리적 1초 간격 40 → 80 ns. 기존 10초 검사 시 00:05
수정 실행	431 ns에서 time=00:05로 FAIL. 기존 기대값 00:10과 차이를 검출하였다.
복구 실행	CLK_HZ=2 복구 후 LAB3_MMSS_PASS checks=7; 종료 144,031 ns. 기존 실행과 일치.

보드에서 확인할 입력과 출력

seg_data[0:7]의 핀은 H2/J7/J3/J1/E4/
E2/F5/F1, seg_com[0:7]은 K5/K3/K1
/L6/G3/G1/H6/H4이다. 자리 선택은
LOW가 활성인 f7, fb, fd, fe를 사용한다.
분·초 자리 순서, 구분점, 깜빡임, 1초 간격
및 59:59 순환을 확인한다. 계수기
PASS만으로 표시 배선과 스캔 동작까지
검증되지 않는다.

7. 실험 6 Character LCD 제어

회로 목적

HD44780 호환 LCD를 8비트 쓰기 전용
방식으로 초기화하고 첫째 줄에 FPGA
LAB3, 둘째 줄에 LCD CONTROLLER를
표시한다. 명령과 문자 데이터의 구분 및
Enable 타이밍을 이해한다.

블록 흐름과 동작 원리

tick 생성 → 전원 대기 → index별 명령.문자 선택 → setup → E HIGH → E LOW → 대기 lcd_rw=0으로 쓰기만 수행한다.

RS=0은 명령, RS=1은 문자 데이터이다.
먼저 데이터와 RS를 설정하고 E를 HIGH로 만든 다음 LOW로 내려 전송을 완료한다. 일반 명령과 화면 지우기 명령에는 서로 다른 대기 시간을 둔다.
40바이트를 보낸 후 index=6으로 돌아가 주소와 문자열을 다시 전송한다.

파라미터 계산

항목	계산과 의미
기준 tick	500클록 × 20 ns = 10 μs, 카운터 9비트
전원 안정 대기	2,000 tick = 20 ms
일반 / clear 대기값	4 / 200 tick = 40 μs / 2 ms
E setup / HIGH	각 1 tick = 10 μs, phase 전이 시간 별도
TB 가속 조건	tick 2클록, POWER=3, NORMAL=1, CLEAR=2

상태와 타이밍

조건	동작
index 0~6, RS=0	38, 38, 38, 0c, 06, 01, 80
index 7~22, RS=1	FPGA LAB3 + 공백 7개
index 23, RS=0	c0: 둘째 줄 시작 주소
index 24~39, RS=1	LCD CONTROLLER + 공백 2개

RTL과 TB 및 XDC의 역할

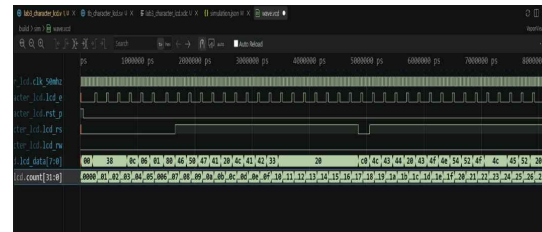
lab3_character_lcd.v가 합성 top으로
명령-문자 ROM에 해당하는 선택 로직과
전송 상태기를 구현한다.

tb_character_lcd.sv의 simulation top은 tb_character_lcd이며 E 하강 에지에서 40바이트의 데이터, RS, RW를 비교한다. XDC는 제어선 3개와 데이터선 8개를 지정한다.

TB 자극과 기대 결과

입력과 검사 조건	기대 결과
초기 명령 7바이트	RS=0, 명령 순서 일치
첫째 줄:주소·둘째 줄	총 40바이트의 값과 RS 일치
전체 전송	RW=0, bytes=40

VS Code 파형 분석



lcd_rw는 계속 LOW이며 초기 명령 구간에서 lcd_rs=0이다. lcd_data의 38이 길게 보이는 것은 같은 초기화 명령을 연속 전송하기 때문이다. 80 다음 46, 50, 47, 41은 ASCII의 F, P, G, A이다. 첫째 줄 데이터에서 RS가 HIGH가 되고, c0 명령에서 잠시 LOW로 바뀐 뒤 둘째 줄 데이터에서 다시 HIGH가 된다. 제공 화면은 count=27(16진수 39)까지 보이며 마지막 전송 완료는 재실행의 bytes=40으로 확인하였다.

Icarus PASS 실행 기록



수정 실험과 복구 결과

lab3_character_lcd.v에서 index=24의
byte_data="L"만 byte_data="M"으로
변경했다. ASCII 데이터는 4c에서 4d로
바뀌며 실제 LCD에서는 MCD
CONTROLLER로 표시될 것으로 예상하였다.
TB의 원래 기대 문자열은 유지했다.

구분	결과
예상	index=24에서 RS=1, 데이터 4c → 4d

구분	결과
수정 실행	5,130 ns에서 index=24 rs=1 data=4d로 FAIL. 문자 변경을 검출하였다.
복구 실행	문자 L 복구 후 LAB3_LCD_PASS bytes=40; 종료 8,130 ns. 기준 실행과 일치.

보드에서 확인할 입력과 출력

lcd_e=A6, lcd_rs=G6, lcd_rw=D6이다.
 lcd_data[0:7]은 A4/B2/C3/D4/A2/
 C5/C1/D1이다. 5 V/3.3 V 결선과 공통
 GND 및 대비 조절을 확인한 후 두 줄
 문자열을 관찰한다. 기능 TB의 바이트
 순서 PASS와 별도로 실제 LCD의 전원
 안정·명령 실행 시간·문자 표시를 확인한다.

8. 실험 7 PC FPGA UART ECHO

회로 목적

9600 baud, 8N1 방식으로 PC에서 받은
 한 바이트를 그대로 송신하고 마지막
 수신값을 LED에 표시한다. 비동기 직렬
 통신의 비트 타이밍과 ready/valid 연결을
 이해한다.

블록 흐름과 동작 원리

uart_rxd → 2단 동기화 → uart_rx →
 rx_data-valid → LED 저장 / TX 요청 →
 uart_tx

UART의 유틸 상태는 HIGH이며 LOW인
 start bit 뒤에 데이터 8비트를 LSB부터
 전송하고 HIGH인 stop bit로 끝낸다.
 수신기는 start bit의 중앙을 확인한 뒤 각
 데이터 비트를 샘플링한다. 송신기는

ready일 때 valid와 data를 받아 프레임을
 보낸다. 수신 시 LED는 갱신되지만 TX가
 바쁘면 에코 요청을 저장하는 FIFO가 없어
 해당 바이트의 송신은 누락될 수 있다.

파라미터 계산

항목	계산과 의미
분주값	$DIV = (50,000,000 + 9,600/2) / 9,600 = 5,208$
1비트 시간	$5,208 \times 20 \text{ ns} = 104.16 \mu\text{s}$
실제 baud와 오차	약 9,600.614 baud, 목표 대비 약 +0.0064%
8N1 한 프레임	start 1 + data 8 + stop 1 = 10비트, 1.0416 ms
TB 가속 조건	CLK_HZ=800, BAUD=100, DIV=8 → 160 ns/bit

상태와 타이밍

조건	동작
유틸 / start	RX-TX=1 / start=0
데이터	bit 0부터 bit 7까지 전송
stop 정상 / 비정상	valid=1 / framing_error=1

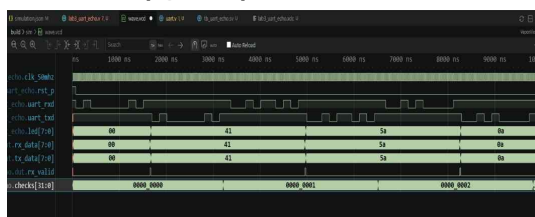
RTL과 TB 및 XDC의 역할

lab3_uart_echo.v가 합성 top이며
 uart.v의 uart_rx·uart_tx를 연결하고 마지막
 수신값을 저장한다. tb_uart_echo.sv의
 simulation top은 tb_uart_echo로 독립
 직렬 모델을 사용해 에코와 LED를
 비교한다. XDC는 RX, TX, LED 및 20 ns
 클록 제약을 지정한다.

TB 자극과 기대 결과

입력과 검사 조건	기대 결과
0x41 입력	A 에코, LED=41
0x5A 입력	Z 에코, LED=5a
0x0A 입력	LF 에코, LED=0a, 최종 checks=3

VS Code 파형 분석



rx_data, tx_data와 led가 00에서 41, 5a, 0a 순서로 바뀐다. 각 수신 완료 시 rx_valid가 짧게 발생하고 이후 uart_txd에 해당 바이트의 직렬 프레임이 나타난다. 0x0A는 줄바꿈 제어문자 LF이다. 수신과 송신에는 처리 지연이 있으므로 RX와 TX 파형이 같은 시각에 완전히 겹치지 않는다. 제공 화면 끝에서 checks가 3에 도달하며 재실행도 checks=3으로 종료하였다.

Icarus PASS 실행 기록



수정 실험과 복구 결과

tb_uart_echo.sv의 DUT

인스턴스에서 .BAUD(100)만 .BAUD(200)으로 변경했다. 외부 직렬 모델의 DIV=8과 데이터 기대값은 유지하였다. DUT의 비트 시간은 80 ns, 외부 모델은 160 ns가 되어

통신 속도 불일치로 기존 에코 검사가
실패할 것으로 예상하였다.

구분	결과
예상	DUT 1비트 160 → 80 ns, 외부 모델 160 ns 유지. 속도 불일치로 에코 오류 예상.
수정 실행	3,230 ns에서 FATAL: echo=fa expected=41. 첫 수신 에코의 불일치를 검출하였다.
복구 실행	BAUD=100 복구 후 LAB3_UART_ECHO_PASS checks=3; 종료 9,970 ns. 기준 실행과 일치.

보드에서 확인할 입력과 출력

uart_rxd=C6, uart_txd=F6이며 LED 핀은 실험 1과 같다. PC 터미널을 9600 8N1, local echo off로 설정한다. 한 문자씩 입력하여 반환 문자 1개와 LED 수신값을 확인한다. TB의 framing_error 검사는 마지막 시점의 신호를 확인하므로 모든 과거 오류 펄스를 누적 감시하는 검사는 아니다. 연속 수신 시의 데이터 손실 여부는 추가 확인이 필요하다. UART 연결은 TX와 RX를 교차하고 논리 전압과 공통 GND를 확인한다.

9. 참고문헌

전전설 LAB3 교안